SIMATS ENGINEERING

ASSESSMENT TOOL 1: Knowledge Representation Design Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO3: *Implement propositional and first-order logic using resolution, unification, and inference techniques for knowledge representation and reasoning. (BTL 3)*

Assessment Tool Weightage: 40 %

Total Marks: 100 No. of Questions: 5 Time: 60 Mins

Annexure A

Knowledge Representation Design Assessment

Code of Conduct:

I **S.Roshni (192525185)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: S.Roshni

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Engineering and Knowledge Base Design	15%	
3. Propositional Logic Representation	10%	
4. First-Order Logic Representation	15%	
5. Unification and Resolution	15%	
6. Forward and Backward Chaining	15%	
7. Inference and Reasoning Accuracy	10%	
8. System Architecture / Representation Diagram	5%	
9. Prototype Implementation and Results	5%	
Total	100%	

Signature of the Faculty: _____ Total Marks Awarded: _____

Assessment Tool 1 – Knowledge Representation Design

Course: Artificial Intelligence and Expert Systems (MLA01)

CO3 – Knowledge Representation, Inference and Reasoning

The assessment requires answering one question and evaluates propositional logic, First-Order Logic (FOL), knowledge-base design, unification, forward/backward chaining, resolution, inference accuracy, and system architecture.

Question selected: Q1 – Smart Healthcare Diagnosis System, because it allows all the required CO3 concepts to be demonstrated clearly.

Q1. Smart Healthcare Diagnosis System

Problem Analysis

The objective is to develop an AI-based healthcare diagnosis system that uses patient symptoms to infer possible diseases and recommend appropriate actions.

The given knowledge is:

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.
- Fever + cough → Flu.
- Flu + body pain → Viral infection.
- Viral infection → Rest is advised.

Entities

Entity	Description
Ravi	Patient
Fever	Symptom
Cough	Symptom
BodyPain	Symptom
Flu	Disease / condition
ViralInfection	Condition
Rest	Recommendation

1. Propositional Logic Representation

Let the propositions be:

F = *Ravi has fever*

C = *Ravi has cough*

B = *Ravi has body pain*

L = *Ravi has flu*

V = *Ravi has viral infection*

R = *Ravi should be advised to rest*

Facts

F, C, B

Rules

Rule 1: If Ravi has fever and cough, then Ravi has flu.

$R1: (F \land C) \rightarrow L$

Rule 2: If Ravi has flu and body pain, then Ravi has viral infection.

$R2: (L \land B) \rightarrow V$

Rule 3: If Ravi has viral infection, then Ravi should rest.

$R3: V \rightarrow R$

Complete Propositional Knowledge Base

$KB = \{ F, C, B, (F \land C) \rightarrow L, (L \land B) \rightarrow V, V \rightarrow R \}$

2. First-Order Logic (FOL) Representation

Let:

Fever(x): x has fever

Cough(x): x has cough

BodyPain(x): x has body pain

Flu(x): x has flu

ViralInfection(x): x has viral infection

NeedsRest(x): x should be advised to rest

Facts

Fever(Ravi), Cough(Ravi), BodyPain(Ravi)

Rule 1

For every patient, fever and cough imply flu:

$\forall x \, (Fever(x) \land Cough(x) \rightarrow Flu(x))$

Rule 2

For every patient, flu and body pain imply viral infection:

$\forall x \, (Flu(x) \land BodyPain(x) \rightarrow ViralInfection(x))$

Rule 3

For every patient, viral infection implies need for rest:

$\forall x \, (ViralInfection(x) \rightarrow NeedsRest(x))$

3. Knowledge Base

The knowledge base contains facts and rules expressed in FOL.

Facts

Fever(Ravi), Cough(Ravi), BodyPain(Ravi)

Rules

Rule	FOL Representation
R1	∀ x (Fever(x) ∧ Cough(x) → Flu(x))
R2	∀ x (Flu(x) ∧ BodyPain(x) → ViralInfection(x))
R3	∀ x (ViralInfection(x) → NeedsRest(x))

Therefore:

KB = { Fever(Ravi), Cough(Ravi), BodyPain(Ravi) } ∪ { R1, R2, R3 }

This structure satisfies the requirement to construct a knowledge base containing facts and rules.

4. Unification

Unification makes two logical expressions identical by finding suitable substitutions.

Example 1

Consider:

Fever(x) and Fever(Ravi)

They can be unified using:

Substitution: {x / Ravi}

After substitution:

Fever(Ravi) ≡ Fever(Ravi)

Therefore, they successfully unify.

Example 2

Consider the rule:

Flu(x) ∧ BodyPain(x) → ViralInfection(x)

and the facts:

Flu(Ravi), BodyPain(Ravi)

The required substitution is:

{x / Ravi}

After substitution:

Flu(Ravi) ∧ BodyPain(Ravi) → ViralInfection(Ravi)

Hence, ViralInfection(Ravi) is derived.

Most General Unifier (MGU)

$$MGU = \{x / Ravi\}$$

5. Forward Chaining

Forward chaining starts from known facts and repeatedly applies rules to derive new facts.

Initial Facts

$$Fever(Ravi), Cough(Ravi), BodyPain(Ravi)$$

Step 1

Apply Rule 1:

$$Fever(Ravi) \land Cough(Ravi) \rightarrow Flu(Ravi)$$

Both conditions are true. Therefore, Flu(Ravi) is derived.

Step 2

Now we have Flu(Ravi) and BodyPain(Ravi). Apply Rule 2:

$$Flu(Ravi) \land BodyPain(Ravi) \rightarrow ViralInfection(Ravi)$$

Therefore, ViralInfection(Ravi) is derived.

Step 3

Apply Rule 3:

$$ViralInfection(Ravi) \rightarrow NeedsRest(Ravi)$$

Therefore, NeedsRest(Ravi) is derived.

Forward-Chaining Table

Step	Known Information	Rule Applied	New Fact
0	Fever(Ravi), Cough(Ravi), BodyPain(Ravi)	—	—
1	Fever(Ravi) + Cough(Ravi)	R1	Flu(Ravi)
2	Flu(Ravi) + BodyPain(Ravi)	R2	ViralInfection(Ravi)
3	ViralInfection(Ravi)	R3	NeedsRest(Ravi)

Final Forward-Chaining Result:

$$KB \vdash NeedsRest(Ravi)$$

6. Backward Chaining

Query:

$$NeedsRest(Ravi)\ ?$$

Backward chaining starts with the goal and works backward toward known facts.

Step 1

Goal: NeedsRest(Ravi). Find a rule that concludes this:

R3: ViralInfection(x) → NeedsRest(x)

Therefore, the new sub-goal is ViralInfection(Ravi).

Step 2

To prove ViralInfection(Ravi), use:

R2: Flu(x) ∧ BodyPain(x) → ViralInfection(x)

New sub-goals: Flu(Ravi) and BodyPain(Ravi).

Step 3

BodyPain(Ravi) is already a fact. To prove Flu(Ravi), use:

R1: Fever(x) ∧ Cough(x) → Flu(x)

New sub-goals: Fever(Ravi) and Cough(Ravi). Both are known facts.

Therefore Flu(Ravi) is proved, then ViralInfection(Ravi) is proved, then NeedsRest(Ravi) is proved.

Backward-Chaining Proof Chain

Fever(Ravi) ∧ Cough(Ravi) ⬆ *Flu(Ravi) ∧ BodyPain(Ravi)* ⬆ *ViralInfection(Ravi)* ⬆ *NeedsRest(Ravi)*

All required facts are available. Therefore, KB ⊢ NeedsRest(Ravi).

7. Resolution

We will prove the query NeedsRest(Ravi) using resolution, which works by proof by contradiction. The negation of the query is added to the knowledge base.

Step 1: Relevant Rules in CNF

Rule 1: Fever(x) ∧ Cough(x) → Flu(x)

CNF: ¬Fever(x) ∨ ¬Cough(x) ∨ Flu(x)

Rule 2: Flu(x) ∧ BodyPain(x) → ViralInfection(x)

CNF: ¬Flu(x) ∨ ¬BodyPain(x) ∨ ViralInfection(x)

Rule 3: ViralInfection(x) → NeedsRest(x)

CNF: ¬ViralInfection(x) ∨ NeedsRest(x)

Step 2: Facts and Negated Query

Facts: Fever(Ravi), Cough(Ravi), BodyPain(Ravi)

Negated query: ¬NeedsRest(Ravi)

Step 3

From Fever(Ravi) and (¬Fever(x) ∨ ¬Cough(x) ∨ Flu(x)) with {x/Ravi}, we obtain:

¬Cough(Ravi) ∨ Flu(Ravi)

Resolve with Cough(Ravi):

Flu(Ravi)

Step 4

From Flu(Ravi) and (¬Flu(x) ∨ ¬BodyPain(x) ∨ ViralInfection(x)) with {x/Ravi}, we get:

$$\neg BodyPain(Ravi) \vee ViralInfection(Ravi)$$

Resolve with BodyPain(Ravi):

$$ViralInfection(Ravi)$$

Step 5

From ViralInfection(Ravi) and (¬ViralInfection(x) ∨ NeedsRest(x)) with {x/Ravi}, we derive:

$$NeedsRest(Ravi)$$

But we initially assumed ¬NeedsRest(Ravi). Resolving NeedsRest(Ravi) with ¬NeedsRest(Ravi) produces the empty clause:

$$\square \ (empty\ clause\ /\ contradiction)$$

Therefore, the negated query is false. Hence, KB ⊨ NeedsRest(Ravi), and Ravi should be advised to rest.

8. Final Inference

The complete reasoning chain is:

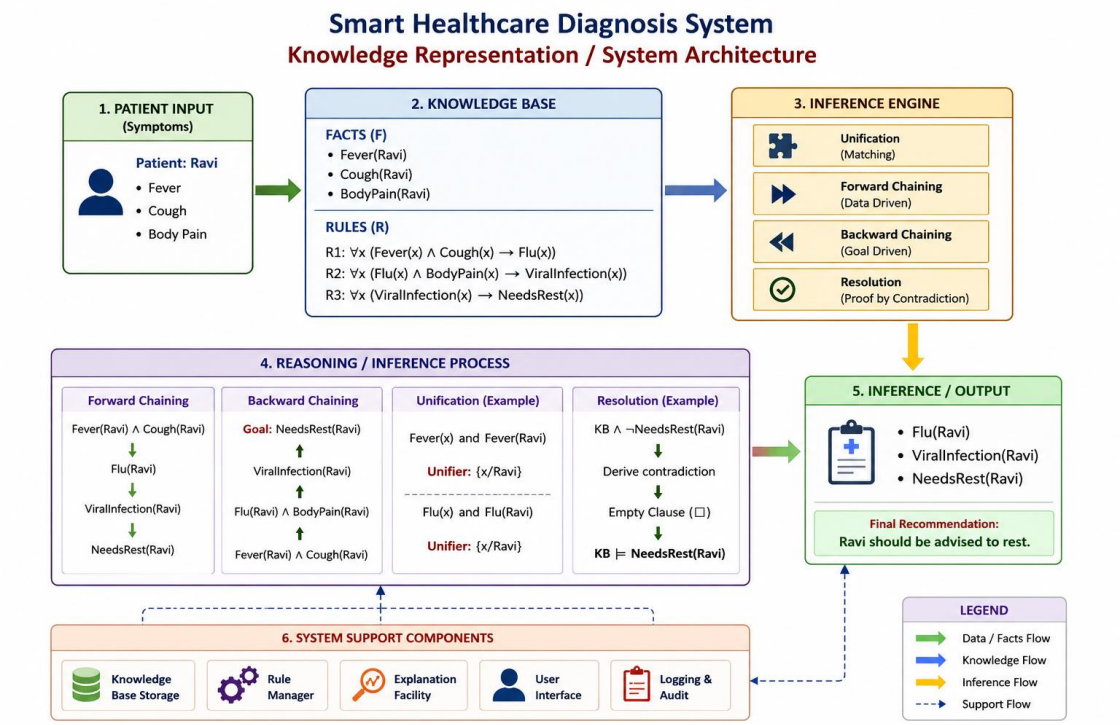
$$Fever(Ravi) \wedge Cough(Ravi) \rightarrow Flu(Ravi) \rightarrow (with\ BodyPain(Ravi)) \rightarrow ViralInfection(Ravi) \rightarrow NeedsRest(Ravi)$$

Final Diagnosis:

The system does not claim a medically confirmed diagnosis; it produces the logical inference supported by the supplied knowledge base — namely that Ravi has flu, has a viral infection, and needs rest.

9. Knowledge Representation / System Architecture

The architecture below shows the flow from patient input through the knowledge base and inference engine to the final recommendation, along with the reasoning process and supporting system components.



This architecture includes the knowledge base, inference engine, reasoning process, and output, matching the assessment's diagram criterion.

10. Comparison of Inference Techniques

Technique	Direction	Application in this Problem
Unification	Matching	Matches Fever(x) with Fever(Ravi) etc.
Forward Chaining	Facts → Goal	Symptoms → Flu → Viral infection → Rest
Backward Chaining	Goal → Facts	Rest → Viral infection → Flu → Symptoms
Resolution	Contradiction-based	Proves that Ravi needs rest
FOL	Predicate-based representation	Represents general patient rules
Propositional Logic	Boolean representation	Represents Ravi-specific facts and rules

11. Final Answer Summary

Component	Result
Patient	Ravi
Symptoms	Fever, cough, body pain
First inference	Flu
Second inference	Viral infection
Final recommendation	Rest

Component	Result
Unifier	{x / Ravi}
Forward chaining	Proves NeedsRest(Ravi)
Backward chaining	Proves NeedsRest(Ravi)
Resolution	Empty clause obtained (contradiction)
Final inference	Ravi should be advised to rest

Conclusion

The Smart Healthcare Diagnosis System represents medical knowledge using Propositional Logic and First-Order Logic. The knowledge base stores Ravi's symptoms and diagnostic rules. Unification matches variables with patient constants, while forward chaining derives the condition from symptoms. Backward chaining starts with the recommendation and verifies the supporting symptoms. Resolution proves the query by contradiction. Thus, all reasoning methods consistently lead to the conclusion:

Ravi should be advised to rest.

This solution directly covers the eight tasks specified in Question 1 of the assessment: propositional logic, FOL, knowledge base design, unification, forward chaining, backward chaining, resolution, and system architecture.

Evaluation Rubrics

Criterion	Weightage (%)	Excellent	Good	Average	Poor
Problem Analysis and Understanding	15%	13–15% – Clearly understands and analyzes the real-world problem, objectives, entities, facts, constraints, and expected inference.	10–12% – Understands the problem with minor omissions.	7–9% – Shows partial understanding with some important elements missing.	0–6% – Poor or incorrect understanding of the problem.
Knowledge Engineering & Knowledge Base Design	15%	13–15% – Develops a complete, consistent, and well-structured knowledge base with appropriate facts, predicates, entities, and rules.	10–12% – Knowledge base is well designed with minor inconsistencies or omissions.	7–9% – Basic knowledge base is developed but several facts/rules are missing or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
Propositional & First-Order Logic Representation	20%	17–20% – Correctly represents the domain using propositions, predicates, variables, constants, quantifiers, and logical connectives with excellent accuracy.	13–16% – Logic representation is mostly correct with minor errors.	9–12% – Provides basic representations but has several logical or syntactic errors.	0–8% – Logic representation is incorrect, incomplete, or missing.
Unification & Resolution	15%	13–15% – Correctly performs substitutions, identifies the MGU, and applies resolution systematically to derive valid conclusions.	10–12% – Applies unification and resolution correctly with minor errors.	7–9% – Demonstrates partial understanding but misses important steps or contains errors.	0–6% – Unable to correctly apply unification or resolution.
Forward Chaining & Backward Chaining	15%	13–15% – Correctly applies both forward and backward chaining with clear step-by-step reasoning and appropriate rule selection.	10–12% – Correctly applies both methods with minor errors or omissions.	7–9% – Demonstrates partial understanding of one or both chaining methods.	0–6% – Incorrectly applies or fails to demonstrate chaining techniques.
Inference & Reasoning Accuracy	10%	9–10% – Produces logically valid conclusions and clearly explains how facts and rules lead to the final inference.	7–8% – Conclusions are mostly correct with minor reasoning gaps.	5–6% – Some correct inferences but reasoning is incomplete or unclear.	0–4% – Conclusions are incorrect, unsupported, or missing.
Knowledge Representation Diagram / System Architecture	5%	5% – Provides a complete, accurate, well-labelled diagram showing knowledge base, inference engine, reasoning process, and output.	4% – Diagram is mostly correct with minor omissions.	3% – Diagram is partially complete or lacks clarity.	0–2% – Diagram is missing or incorrect.
Originality, Critical Thinking & Presentation	5%	5% – Demonstrates excellent independent thinking, appropriate real-world application, comparison of reasoning approaches, and clear presentation.	4% – Demonstrates good reasoning and clear presentation.	3% – Shows limited critical thinking and basic presentation.	0–2% – Shows little originality, weak reasoning, or poor presentation.